

What is MeTTa?

The rule-based AI language, explained from zero, for someone who knows a little C.

```
; facts
(parent Tom Bob)
(parent Bob Ann)

; a rule: grandchild = child of a child
(= (grandchild $p)
  (match &self (parent $p $c)
    (match &self (parent $c $g) $g)))

!(grandchild Tom)      →  [Ann]
```

Nobody told the program that Ann is Tom's grandchild. It worked that out from two facts and one rule. That is MeTTa.

MeTTa in one minute

MeTTa (short for *Meta Type Talk*) is a programming language made by **SingularityNET** for building AI that **reasons with rules**. It is the core language of **OpenCog Hyperon**, their project aimed at general-purpose AI (AGI).

- You give MeTTa **facts** (things that are true) and **rules** (how to work out new things).
- When you ask a question, MeTTa **finds the answer by matching** your question against those facts and rules.
- Because the facts and rules are written out plainly, you can always see **why** it gave an answer.
- A MeTTa program can **add or remove its own facts and rules while it runs**, so it can learn.

One-line summary

C is a language for telling a computer **what to do**. MeTTa is a language for telling a computer **what is true**, then letting it **work things out**.

How MeTTa is different from C

You already know some C, so here is the comparison side by side.

	C	MeTTa
Style	Step-by-step instructions (imperative)	Facts + rules + questions (declarative, symbolic)
How you get an answer	You write the loop / if-else that computes it	You ask a question; MeTTa pattern-matches to find it
Syntax	<code>int x = 5; printf(...);</code>	Everything in brackets: (parent Tom Bob)
Data	Variables, arrays, structs	A knowledge base full of “atoms” and expressions
Variables	x, declared with a type	\$x, a blank that MeTTa fills in
Number of answers	A function returns one value	A question can return several answers at once
Changing the program while running	Not really	Normal: add-atom / remove-atom
Explaining a result	Hard; logic is buried in code	Easy; the rules that produced it are visible

If you've heard of **Lisp** (all the brackets) or **Prolog** (facts and rules), MeTTa borrows ideas from both.

The building blocks

Six things you need to recognise to read any MeTTa code.

Piece	What it is	Example
Atom	The smallest unit: a word, number or name.	Tom, 5, parent
(...)	An expression : atoms grouped in brackets. All code and data are expressions.	(parent Tom Bob)
\$x	A variable . Any name starting with \$ is a blank to be filled.	(parent Tom \$x)
&self	The program's own knowledge base (called a "space"), where facts live.	(match &self ...)
=	Defines a rule or function : "this can be rewritten as that".	(= (square \$x) (* \$x \$x))
!	Run this now and show the result. Lines without ! are just stored.	!(square 5)
;	A comment , ignored by MeTTa (like // in C).	; my note

Prefix notation

The operator always comes **first** inside the brackets. So $2 + 3$ in C is $(+ 2 3)$ in MeTTa, and $a < b$ is $(< a b)$.

Learn by example

Six small programs, each introducing one idea. The green line under each is MeTTa's real output.

Example 1 · Facts and a question

```
METTA
; facts
(parent Tom Bob)
(parent Bob Ann)

; question: who is Tom's child?
!(match &self (parent Tom $x) $x)

[Bob]
```

match takes three things: **where** to look (&self), a **pattern** to look for ((parent Tom \$x)), and **what to return** (\$x). MeTTa finds (parent Tom Bob), sees that \$x must be Bob, and returns it.

Example 2 · More than one answer

```
METTA
(parent Tom Bob)
(parent Tom Liz)

!(match &self (parent Tom $x) $x)

[Bob, Liz]
```

In C a function returns one value. In MeTTa a question returns **every** match it finds. You didn't write a loop; MeTTa searched the whole knowledge base for you.

Example 3 · A rule that works things out

```
METTA
(parent Tom Bob)
(parent Bob Ann)

(= (grandchild $p)
  (match &self (parent $p $c)
    (match &self (parent $c $g) $g)))

!(grandchild Tom)

[Ann]
```

Read the rule as: “the grandchild of \$p is: find a child \$c of \$p, then find a child \$g of \$c, and return \$g.” The fact “Ann is Tom's grandchild” was never stored. MeTTa **derived** it by chaining two facts. This is **reasoning**, and it is the heart of what MeTTa is for.

Example 4 · An ordinary function

```
METTA
(= (square $x) (* $x $x))

!(square 5)

[25]
```

The same `=` used for rules also defines normal functions. Maths works with prefix notation: `(* $x $x)` means $x \times x$.

Example 5 · Making a decision with if

```
METTA
(= (income Priya) 180000)
(= (marks Priya) 88)

(= (decide $s)
    (if (and (< (income $s) 250000) (> (marks $s) 75))
        (Approved $s)
        (Rejected $s)))

!(decide Priya)

[(Approved Priya)]
```

`if` works like C: `(if condition then-part else-part)`. Notice the answer isn't just “true”. It is a meaningful expression, `(Approved Priya)`, which you can extend to carry the *reasons* as well.

Example 6 · The program learns something new

```
METTA
(parent Tom Bob)
(parent Bob Ann)

!(add-atom &self (parent Ann Joe))
!(match &self (parent Ann $x) $x)

[()]
[Joe]
```

`add-atom` puts a new fact into the knowledge base **while the program is running** (the `()` just means “done”). The next question can already use it. `remove-atom` deletes a fact the same way. This is how a MeTTa agent can learn and correct itself.

Why the hackathon uses MeTTa

The two MeTTa Foundation challenges each map directly onto one of MeTTa's superpowers.

MeTTa superpower	Shown in	Hackathon challenge it powers
Transparent reasoning: every answer comes from visible facts and rules, so you can trace why	Examples 3 & 5	1.1 The Glass Box Agent: an agent that makes a decision and explains every step
Self-modification: the program can add, remove and replace its own facts and rules while running	Example 6	1.2 The Agent That Grows Up: an agent that gets corrected and visibly updates its rules

Most modern AI (like chatbots) is a **black box**: it gives an answer but can't truly show the reasoning. MeTTa is a **glass box**: the reasoning *is* the program. That's why SingularityNET calls this "beneficial AI": AI whose decisions people can check, question and trust.

Installing and running MeTTa

MeTTa installs as a Python package called hyperon.

Step 1 · Check your Python version

TERMINAL

```
python --version
```

Important for Windows

The Windows package currently supports **Python 3.8 to 3.12**. There is **no Windows build for Python 3.13 or newer** yet, so if `pip install` fails, that is the likely reason. Install Python 3.12 from python.org and use it for this.

Step 2 · Install

TERMINAL

```
pip install hyperon
```

Step 3 · Run a .metta file

Save the code in a file such as `test.metta`, then run it:

TEST.METTA

```
; test.metta
(= (square $x) (* $x $x))
!(square 5)
```

TERMINAL

```
metta-py test.metta
```

[25]

Step 4 (optional) · Run MeTTa from inside Python

Useful later, when you want a Python program to feed inputs to MeTTa and print a nice explanation:

PYTHON

```
from hyperon import MeTTa
```

```
m = MeTTa()
result = m.run("""
(= (square $x) (* $x $x))
!(square 5)
""")
print(result)
```

[[25]]

Python shows [[25]]: an outer list with one entry per ! line, and inside it the list of answers for that line.

07

Cheat sheet

Keep this page open while you practise.

Code	Meaning
(fact a b)	Store a fact in the knowledge base
!(expr)	Run expr now and print the result
\$x	A variable (blank to be filled)
(= (name \$a) body)	Define a rule or function
(match &self pattern result)	Search the knowledge base for pattern; return result for each match
(if cond then else)	Choose between two results
(and a b) (or a b) (not a)	Logic
(+ a b) (- a b) (* a b) (/ a b)	Maths (operator first)
(< a b) (> a b) (== a b)	Comparisons
!(add-atom &self (fact))	Add a fact while running (learning)
!(remove-atom &self (fact))	Delete a fact while running (correcting)
; text	Comment

What to do next

Practise the language before Sept 30, but not your actual hackathon project.

- Install MeTTa and run all six examples above yourself. Change the names and numbers and predict the output before you run it.
- Make your own family tree with 6–8 people and write rules for sibling, grandparent and cousin.
- Write a tiny decision rule on an unrelated topic (e.g. “which movie should I watch?”) using `if`.
- Try `add-atom` and `remove-atom` and check that later questions change.
- Read the official tutorials and docs at **metta-lang.dev**.

Remember: learning and practising MeTTa now is fine. Build the **actual hackathon project** from Sept 30, during the event.